LLMSmartSec: Smart Contract Security Auditing with LLM and Annotated Control Flow Graph

Viraaji Mothukuri, Reza M. Parizi, James L. Massa
Decentralized Science Lab
College of Computing and Software Engineering, Kennesaw State University
Marietta, GA 30060, USA
vmothuku@students.kennesaw.edu, rparizi1@kennesaw.edu, james.l.massa@gmail.com

2024 IEEE International Conference on Blockchain (Blockchain) | 979-8-3503-5159-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/BLOCKCHAIN62396.2024.00064

***Abstract*—Historically, the complexity of identifying vulnerabilities in smart contracts required human-intensive audits to supplement imprecise automated code scans. The growing smart contract market highlights the urgency for effective automated security auditing. Furthermore, new AI-powered coding assistants can introduce vulnerabilities that evade engineers, and autonomous AI can now write 100% code changes without human oversight. This research introduces LLMSmartSec, a new approach that accurately identifies and fixes smart contract vulnerabilities, leveraging the strengths of machine intelligence to operate at high speed, scale, and without fatigue. To develop LLMSmartSec, we first fine-tuned Open AI GPT-4 to understand Solidity, the programming language of Ethereum blockchain, and to micro-analyze smart contracts holistically from the viewpoints of the developer (LLMDev), auditor (LLMAudit), and ethical hacker (LLMeHack) and to identify vulnerabilities and generate code fixes for them. We used GPT-4 again to store the smart contract code in a Control Flow Graph (CFG) annotated with the code vulnerabilities and their fixes. Finally, we trained an LLMGraphAgent with open-source LLMs on the annotated CFG so that it can run locally, identifying vulnerabilities and fixes without costly calls to GPT-4. This research enhances the security of blockchain projects in Web3 by supplementing a costly human bottleneck with a low-cost automated security tool based upon the innovative creation of an annotated CFG with GPT-4.**

***Index Terms*—Blockchain, Smart Contracts, Security Auditing, AI, Large Language Models (LLMs), OpenAI Assistants, Control Flow Graphs (CFGs).**

I. Introduction

Smart contract [1] technology enables blockchain applications to automate transactions that execute the terms of a contract when certain conditions are met. Smart contracts are deployed on a blockchain network [2], a decentralized ledger that records transactions. They provide high security and trustworthiness to transact and benefit from blockchain applications. These contracts are written in programming languages specific to blockchain platforms; the code defines the rules and consequences of the contract, similar to a traditional legal contract, but in a way that is automatically executed and verified by the blockchain network. The automated actions include and are not limited to transferring funds, issuing tokens, or updating information. The self-executing automated nature of smart contracts eliminates the need for human intervention and intermediaries, reducing costs and delays with a manual, traditional approach. Smart contracts have a wide range of applications in Web3 [3], [4], [5], including Decentralized Finance (DeFi), which provides decentralized financial services over blockchain digital, supply chain management, digital identity, and Non-fungible Tokens (NFTs).

However, smart contracts carry inherent risks because of blockchain's immutability (making it hard to fix mistakes), lack of central authority (no one to intervene if something goes wrong), and potential coding issues (bugs or vulnerabilities). Historical incidents reveal that vulnerabilities in smart contracts can lead to system failures and financial losses. For example, hacks on the Ethereum blockchain have resulted in the theft of millions of dollars in cryptocurrency. Many of these scams and attacks over the years [6], [7], [8] could have been prevented if developers had conducted thorough, smart contract code audits before deployment. Apart from the unintended bugs introduced by developers in a production environment, some hacks and scams occur when developers embed backdoors in the code to steal tokens or cryptocurrency. However, most attacks are carried out by hackers exploiting weaknesses in the smart contract code.

Due to these risks, smart contract audits are crucial. The projected market growth of smart contracts underscores the importance of security audits, with the market expected to grow from $2.14 billion in 2024 to $12.55 billion by 2032 [9]. Audits help build trust in the blockchain ecosystem and ensure contract security and reliability. While there are numerous tools and frameworks for automatically auditing smart contracts, research [10], [11] recommends combining automated audits with mandatory manual audits. This is because current tools often fail to understand the context and intent of complex smart contract code.

Smart contract code is often more complex than other types of code. To effectively audit such complex code, we propose a new approach, *LLMSmartSec*, using large language models (LLMs) that is trained with the collective knowledge of humans involved in the manual auditing process, mimicking their steps. Our approach involves micro-analyzing smart contracts from the perspectives of the developer (LLMDev), auditor (LLMAudit), and ethical hacker (LLMeHack) to identify vulnerabilities and suggest code fixes, which is innovative. Notably, LLMs are ideal for auditing complex code because they have reached, and sometimes exceeded, human-level performance for code and analysis. Numerous studies [12], [13], [14] explore LLMs for code analysis, demonstrating that LLMs can efficiently and accurately handle large volumes of complex code. This efficiency and capability serve as the primary driving force behind our research.

A. Contributions

Our work contributes to the development of secure smart contracts in Web3 and blockchain-based projects. Additionally, we provide new insight into existing tools, processes, and methodologies for smart contract auditing. We also investigate the latest advancements in NLP-based machine learning models that can effectively automate the audit process. The following summarizes our main contributions:

- *LLM-Powered Smart Contract Audit and Fix*: LLMSmartSec leverages state-of-the-art LLM technology to audit smart contracts with better accuracy. Increased accuracy includes better recall, reducing the need for human audits on top of automated audits, as well as better precision, reducing the need for analysts to sift through false positives to find true vulnerabilities. Harnessing the power of Generative Artificial Intelligence (GenAI), we also generate the code fix for vulnerabilities. We trained our model on the Solidity blockchain programming language, insights from

2834-9946/24/$31.00 ©2024 IEEE
DOI 10.1109/Blockchain62396.2024.00064

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

static tools, and embeddings of known code patterns, using existing knowledge from historic hacks and keeping it up-to-date with the expertise of LLMs analysis.

- *Automatically Populated and Annotated Control Flow Graphs for Smart Contracts*: Using GPT-4, we efficiently populate a Control Flow Graph (CFG) with the high-risk portions of analyzed smart contract code, a job that can take 6,000 lines of error-prone code [15] when programmed by hand. Then, we annotate the CFG with the vulnerabilities found by LLM and the fixes that the model generated.
- *Training a Local LLMGraphAgent with Annotated CFG for Efficient and Independent Operation*: The annotated CFG serves as the training data for an LLMGraphAgent (explained in Section III), a locally running agent that eliminates the cost and latency of API calls to Open AI. By training the agent on the annotated CFG, we enable it to operate efficiently and independently without relying on external services/unknown versions of LLMs.

The rest of the paper is organized as follows: Section II reviews related works. Section III presents our proposed approach, LLMSmartSec, detailing its architecture, implementation, and internal mechanisms. Section IV discusses the evaluation results. Finally, Section V provides the conclusion and outlines future work.

II. Related Work

Authors in research work [11] evaluate GPT-4 and Claude LLM's ability to identify vulnerabilities in smart contracts. They suggest that LLMs can be more of an assistant than a replacement for smart contracts, as their evaluation metrics highlight a significant fraction of false positives. The authors emphasize human intelligence and suggest LLM-powered manual audits. In research work [16], authors propose TrustLLM, with Detector, Reasoner, Ranker, and Critic roles, where Detector is used to detect the vulnerabilities, Ranker ranks them on a scale of (1-10), the Reasoner provides the reasoning and the Critic takes criticizes the analyzes and asks Reasoner to re-evaluate. TrustLLM combines fine-tuning and LLM-based agents to audit smart contracts. With majority voting and multiple prompts, they focus on auditing the smart contract and achieve better results than GPT-4. Similar roles of an auditor and a critic are proposed for GPTLens in [17].

Another research work proposed GPTScan [18] to identify the vulnerabilities in smart contacts with an average scanning time of 14.39 seconds (approx.) and a cost of 0.01 USD per thousand lines of Solidity code. As per the authors, GPT3.5 struggles to deliver optimal results with large datasets of smart contract projects.

Authors in [19] propose an automated and cost-effective audit of smart contracts with LLMs (GPT4-turbo) in the research work for compliance with Ethereum Request for Comment (ERC) standards. AuditGPT attempts to identify violations of rules by leveraging the LLM's capability to compare smart contact with their corresponding ERCs. SMARTINV framework with novel Tier of Thought (ToT) prompting approach proposed in research work [20] to automate the detection of bugs in smart contracts. The framework considers code comments and documentation along with the source code to understand the intended behavior of the smart contract. The authors finetuned the base LLaMA-7B with the Parameter Efficient Finetuning (PEFT) technique using low-rank adaptation (LoRA) and quantized it to 8-bit.

The research paper [15] empirically assesses nine automated analysis tools for Ethereum smart contracts. These are Honeybadger, Maian, Manticore, Mythril, Osiris, Oyente, Securify, Slither, Smartcheck. Leveraging two distinct datasets: one, referred to as a smaller dataset in the paper comprising 69 annotated vulnerable smart contracts for precision evaluation, and the second, a more extensive collection of 47,518 contracts. The assessment used SmartBugs, an execution framework developed to integrate and compare the tools. The study stands out due to its extensive scope, containing 428,337 analyses over roughly 564 days and 3 hours. The results revealed that merely 42% of the vulnerabilities in the annotated dataset were detected by all the tools, with Mythril achieving the highest level of accuracy at 27%. In the larger dataset, 97% of the contracts were identified as vulnerable, indicating a high probability of false positives. Additionally, the study observed that only a limited number of vulnerabilities, specifically those falling into two categories, were simultaneously detected by four or more tools. This emphasizes the limitations of current automated analysis tools in effectively identifying vulnerabilities in Ethereum smart contracts. Furthermore, similar studies [21], [22] empirically analyze existing auditing tools for smart contract vulnerabilities, showing similar finding patterns.

Similarly, attempts made with the latest NLP models such as GPT [23], [24], [17], [25],[26] fail to understand the complexity of the smart contract code and identify the security risks in the code with higher accuracy rates. This emphasizes the limitations of current automated analysis tools and ML-based solutions in effectively identifying vulnerabilities in Ethereum smart contracts. Tables I and II list the related work of smart contract audit tools and works, including research from static and dynamic analysis to machine learning and LLM-based approaches.

TABLE I: Literature Review of Static, Dynamic, and Machine Learning Tools for Smart Contract Audits

Tool/Research	Description
Static & Dynamic Analysis Tools	
Slither [27]	A well-designed static analyzer tool; It finds bugs, suggests code optimizations, and increases code understanding by Analyzing the reduced instruction set of the code.
Honeybadger [28]	Symbolic execution tool, designed specifically to identify honeypot smart contracts.
Maian [29]	Symbolic analysis of bytecode to identify vulnerabilities and provide the code to exploit it.
Osiris [30]	Combines symbolic execution with taint analysis.
Smartcheck [31]	Another static analyzer tool that analyzes the code without executing it.
Oyente [32]	A symbolic execution tool that works on the smart contract's bytecode.
Mythril [33]	Combines concolic analysis, taint analysis, and control flow analysis.
Securify [34]	Automated scanner that can be integrated into the development pipeline.
Manticore [35]	Symbolic execution tool that exhaustively explores all the possible execution paths.
Machine Learning Tools	
[36]	Control flow graph (CFG) is built using smart contract opcode. Graph Convolutional Network (GCN) is trained with CFG and Vulnerability labels.
[37]	A degree-free graph convolutional neural network (DR-GCN) and a novel temporal message propagation network (TMP) are proposed to identify vulnerabilities using Graph Neural Networks.
[38]	Utilizes Bytecode.
[39]	Combines the best of GCN and Bi-LSTM models.
[40]	Converts the source code of the smart contract into UML diagrams to visualize the graphical representation of the smart contract.
[41]	Graph Db [42] for identifying smart contract vulnerabilities with GraphCodeBert.
[43]	Lightening Cat, uses slither for labeling Code Bert
[44]	Builds a graph from smart contract code by extracting Node features, Semantic features, and Grammatical features. Uses [45] dataset to evaluate the approach.
[23]	Automated Security Analysis of Smart Contracts with graph traversal technique.

III. Proposed Approach: LLMSmartSec

A. Architecture

Figure 1 overviews our proposed approach LLMSmartSec. We start with creating AI assistants with instructions to analyze the smart contract. An annotated CFG is built using existing code patterns with the schema (as illustrated in Figure 3b) that contains metadata of smart contracts with the labels of vulnerability and insights from LLM. LLMGraphAgent is leveraged for every new code review process and is updated with the insights from the new review from the LLMSmartSec pipeline.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

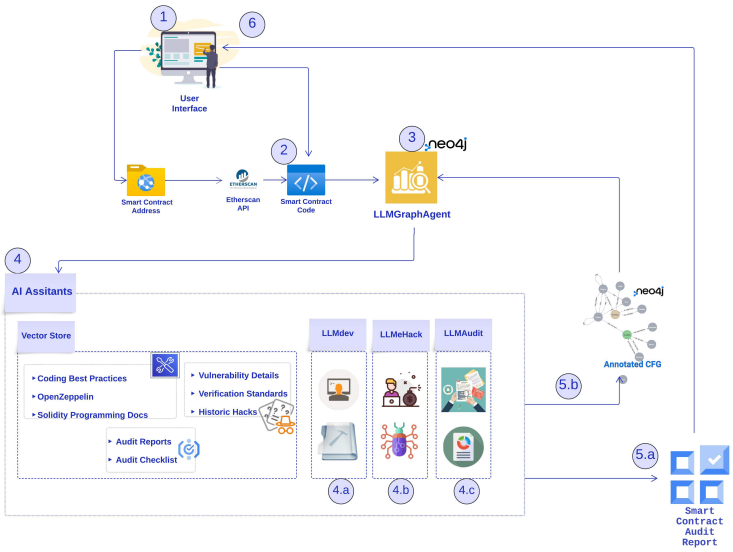


Fig. 1: LLMSmartSec Architecture

TABLE II: Literature Review of LLMs for Smart Contract Audits

Tool/Research	Description
[24]	Smart Contract Armour: Combines Rule-based engine with ChatGPT API. A simplistic rule engine is used, and ChatGPT is leveraged for complex questions.
[46]	G-Scan Vulnerability scan with GPT3.5.
[47]	Code Vulnerabilities with LLMs not specific to smart contracts.
[48]	Analysis of chatGPT efficiency in identifying the vulnerabilities in smart contract, employs prompts to chatGPT directly to analyze its capability in identifying issues with the code.
[49]	LLM4Fuzz leverages LLMs to prioritize the fuzzing, specialized prompt to extract insights from the smart contract code.
[17]	GPTLens: proposes critic and auditor roles to audit the smart contract code for vulnerabilities.
[18]	Leverages GPT3.5 to scan the code in minimum time.
[11]	LLMs for 50 compromised smart contracts False positives might be relying on the trained data of publicly available info on compromised smart contracts information.

The user interface allows users to enter a smart contract address or code snippet. The Etherscan API fetches the code data from the provided address for smart contracts. The extracted code, along with the insights into existing patterns from the LLMGraphAgent, is shared with the three assistants, and the response from the three assistants is consolidated to generate the audit report. We empower each assistant with vector store, that can help in analyzing the smart contract code. Below is the summary of the design and implementation steps of the proposed approach, as shown in Figure 1.

- Build LLMGraphAgent on a graph database using the existing information on known vulnerabilities in smart contract code.
- Build a pipeline to review the smart contract code and generate Cypher commands with insights from smart contract code analysis.
- Execute the Cypher commands generated for every smart contract address on the graph database to generate an Annotated Control Flow Graph.
- Design the prompts and instructions for AI assistants to review the new smart contract code from three views: LLMDev, LLMeHack, and LLMAudit.
- Empower each LLM (LLMDev, LLMeHack, and LLMAudit) with role-specific and relevant document information in the form of a vector store database for reference during inference.
- Build the UI that enables the user to share either the smart contract address or the smart contract code.
- Build a pipeline to collect the insights from each LLM view and provide it to Summary LLM to generate the report.
- Review the generated report for identified vulnerabilities, potential risks, and recommendations.

Below are the details of the steps of the working aspect of the solution marked in the architecture diagram.

1) User enters the smart contract details on the user interface.
2) Extract the code of the smart contract address.
3) Preprocess the code to remove irrelevant information and match the code pattern with the existing patterns in LLM-GraphAgent. If matched, skip 4a and 4b, and send the analysis summary to 4c to generate the audit report.
4) Else, analyze the code with AI assistants 4a, 4b, and 4c.
5) Generate:
   a) The audit report with observations, recommendations, and mitigation steps.
   b) The annotated CFG and update the LLMGraphAgent with the smart contract's latest pattern, vulnerability, and code details.
6) Display smart contract audit report back to the user.

Algorithm 1, lists the steps of creating base infrastructure with LLMs and APIs and the task orchestrated for automating the smart contract audit report generation in a formal way. According to the algorithm, Lines 1-14 contain the implementation of the "Init" function, which creates and initializes the four assistant LLMs, loads files III-C for each assistant, and creates vector stores for each set of loaded files. The "Init" function returns the initialized LLMs and their corresponding vector stores. Lines 15-16 define a new function called "PreWork," which is responsible for initializing and loading the assistants and vector stores by calling the previously described "Init" function. This function ensures that the necessary components are set up before the main audit process begins. The main function, "Audit Report Generator," is implemented in lines 17-26. This function takes the Smart Contract Code as input and performs the following steps:

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

- Retrieves the Smart Contract Code using the "RetrieveContractCode" function and the provided Smart Contract Address.
- Obtains insights from the Annotated CFG by calling the "ReviewCode" function with the Smart Contract Code as input.
- Utilizes the developer assistant (LLMDev) to review the Smart Contract Code and provide a developer review.
- Employs the ethical hacker assistant (LLMeHack) to scan the Smart Contract Code for potential vulnerabilities.
- Uses the auditor assistant (LLMAudit) to perform an audit based on the Smart Contract Code, developer review, and identified vulnerabilities.

B. LLMGraphAgent

Figure 2 illustrates the annotated control graph creation steps. We use the existing knowledge on smart contract vulnerabilities [45], [50] with the schema visualized in Figure 3b. We further enrich the Annotated Control Flow Graph with insights from LLM on the labeled smart contract code. For every new smart contract code review process, a graph is built with LLM generated Cypher commands and scanned against the existing graph data for similar patterns. Figure 3a represents classic CFG created by static tools, whereas 3b is the annotated CFG for a smart contract code with Reentrancy Vulnerability.

The work of LLMGraphAgent is crucial as it helps to build a knowledge base of known patterns in the code that can be referenced as a secondary memory of LLM for future reviews. Furthermore, if a match is found with existing patterns, the analysis costs with LLMs are saved as the automated process skips the review of LLMDev and LLMeHack and proceeds to the audit report generation step. Prompts/instruction snapshot for generating Cypher queries to build the graph database for $LLMGraphAgent$ is below.

Prompts for Generating Cypher

■ You are an expert smart contract Reviewer with expertise in Neo4j graph database. Your task is to generate Cypher queries that create a comprehensive Annotated Control Graph (KG) representing vulnerability patterns in smart contracts. The KG should capture the following crucial elements:

- Read each line of code and understand the execution paths.
- Please generate Cypher queries that create the necessary nodes, relationships, and properties to build the vulnerabilities pattern Annotated Control Flow Graph. Ensure the queries are properly formatted, efficient, and adhere to Neo4j best practices.
- Provide the generated Cypher queries in a code block format, with each query separated by a blank line for readability.
- ...

Using the schema discussed in the above section, we built a graph agent with the graph data. This is referred to as LLMGraphAgent, the final product of our proposed approach. This empowers the smart contract auditing task with information on existing knowledge of vulnerabilities, recommendations, and approved fixes. This is built with an open-source version of LLMs and is available at zero inference cost.

C. Vector Store

Retrieval-augmented generation (RAG) helps the LLM give accurate info with source attribution and avoid hallucinations by allowing uploading enterprise and domain-specific knowledge. The output can include citations or references, so users can check the original documents if they need more details. This boosts trust and confidence in the GenAI solution. The OpenAI assistants take away a RAG pipeline's complexity but provide similarity

Algorithm 1: LLMSmartSec

Input: Smart Contract Code
Output: LLM Generated Audit Report

```
Function Init():
        /* Create the four Assistants */
    llm_dev ← InitializeLLMDev(model="gpt-4",
      file_search=True, vector_store=True)
    llm_ehack ← InitializeLLMeHack(model="gpt-4",
      file_search=True, vector_store=True)
    llm_audit ← InitializeLLMAudit(model="gpt-4",
      file_search=True, vector_store=True)
    llm_report ←
      InitializeLLMReportGenerator(model="gpt-4",
      file_search=True, vector_store=True)
        /* Load the Assistants */
    dev_files ← LoadFiles("smart_contract_development")
    ehack_files ←
      LoadFiles("smart_contract_vulnerabilities")
    audit_files ← LoadFiles("smart_contract_auditing")
    report_files ← LoadFiles("audit_report_templates")
        /* Vector Store */
    dev_vector_store ← CreateVectorStore(dev_files)
    ehack_vector_store ← CreateVectorStore(ehack_files)
    audit_vector_store ← CreateVectorStore(audit_files)
    report_vector_store ← CreateVectorStore(report_files)
    return llm_dev, llm_ehack, llm_audit, llm_report,
      dev_vector_store, ehack_vector_store,
      audit_vector_store, report_vector_store;
Function PreWork():
      /* Initialize and load assistants and vector
      stores */
    llm_dev, llm_ehack, llm_audit, llm_report,
      dev_vector_store, ehack_vector_store,
      audit_vector_store, report_vector_store ← Init();
Function Audit Report
  Generator(SmartContractCode):
        /* Get Smart Contract Code */
    SmartContractCode ←
      RetrieveContractCode(SmartContractAddress)
        /* Get Insights from Annotated CFG */
    AnnotatdCFGReview ←
      ReviewCode(SmartContractCode)
        /* Get Developer Review */
    DevReview ←
      LLMDev.ReviewCode(SmartContractCode)
        /* Get Vulnerability Review */
    Vulnerabilities ←
      LLMeHack.ScanVulnerabilities(SmartContractCode)
        /* Get Auditor LLM Review */
    AuditReport ← LLMAudit.PerformAudit
    (SmartContractCode, DevReview, Vulnerabilities)
        /* Get Audit Report  */
    FinalAuditReport ← LLMReport.GenerateReport
    (SmartContractCode, DevReview, Vulnerabilities,
      AuditReport)
    return FinalAuditReport;
```


Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

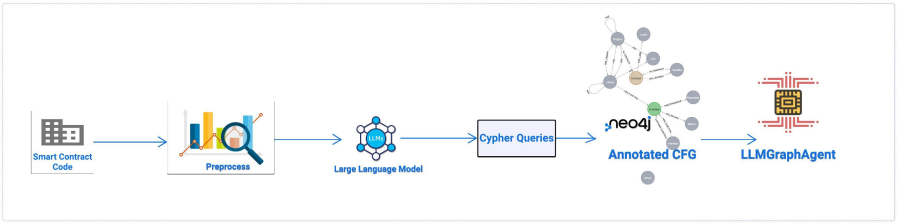
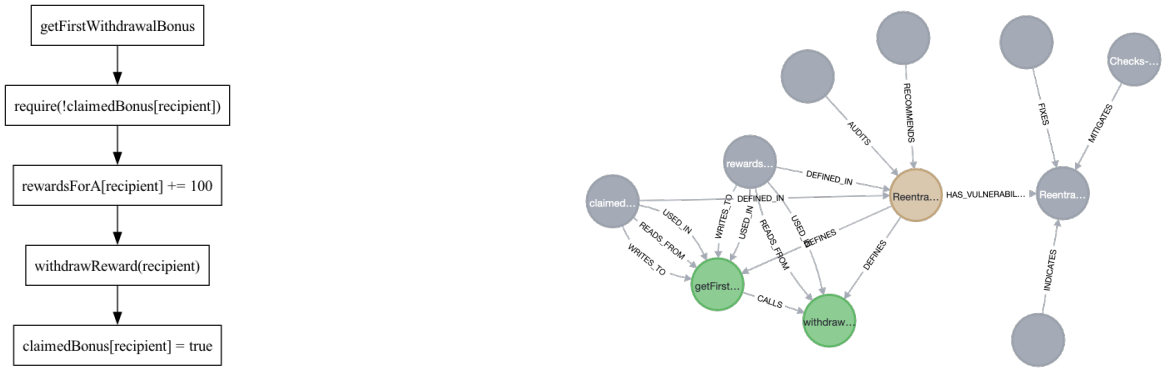


Fig. 2: Workflow for Construction of Annotated CFG (LLMGraphAgent)



(a) Classic CFG

(b) Annotated CFG (LLMGraphAgent)

Fig. 3: Comparison of Classic CFG and Annotated CFG

functionality, which is why we used them in this research. With that said, we create a Vector store for three AI assistants with task-specific documentation. The file search capability of assistants converts the uploaded documents into vector stores for LLM Assistants to retrieve and reference faster while responding to the given prompt. Below is the list of documents we used to convert into vector embeddings (the approach allows uploading more documents as needed).

1) Solidity Programming Specification document [51]
2) Audit Reports Datasets [52]
3) Slither Audited Smart Contracts [50]

The open AI assistants file search tool leverages the vector store and empowers the LLM by uploading the information as files. The uploaded files are automatically parsed, chunks, embedded, and saved in the vector store.

D. Prompts and Instructions

Each assistant is provided with a set of instructions; below are the overviews of the prompts given to the AI assistants.

Prompts to LLMDev

- You are an expert smart contract developer. Your task is to analyze the provided smart contract code from a developer's perspective. Follow the below steps:
  - Read each line of code and understand the execution paths.
  - Assess the impact of the code and share review comments as a peer reviewer
  -

Prompts to LLMeHack

- You are a smart contract eHacker. Your task is to analyze the provided smart contract code for potential risks. Follow the below steps:

1. Identify any severe vulnerabilities in the smart contract that could be exploitable in the real world and beneficial to attackers. Provide a detailed explanation of each identified vulnerability, including:
   - The specific vulnerability type (e.g., reentrancy, integer overflow/underflow, access control issues, Timestamp Dependence, Access Control Vulnerabilities, Front-running Attacks, Denial of Service (DoS) Attacks, Logic Errors, Insecure Randomness, Gas Limit Vulnerabilities, Unchecked External Calls, Uninitialized Storage Pointers, Gas Limit and Loops, Delegate call Attacks)
   - ..

Prompts to LLMAudit

- You are a smart contract auditor. Your task is to provide a detailed report of your findings, including the identified vulnerabilities (if any), their potential impact, recommendations for mitigation, and the overall risk score. If the contract is deemed to be clean and secure, emphasize the positive security measures in place.
  - Report should have detailed information with sections: Introduction, Executive Summary, Smart Contract details, Gas Optimization, and Vulnerabilities categorized as High, Medium, and Low.
  - ...

IV. Evaluation and Results

A. Experimental setup

For the evaluation of our proposed approach, we use Open AI's Assistant API and leverage Code Interpreter, File Search, and Function calling features. For the LLMGraphAgent Graph Database, we leveraged the graph database Neo4J Desktop v1.5.9.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

The entire coding task is implemented in Python3.10. For LLMs we have leveraged gpt-3.5-turbo-0125 [53] and GPT4 [54]. Our development environment is built by utilizing Google Colab Pro paired with a T4 GPU, with 50 GB of RAM and 160 GB of space. Additionally, we utilized a MacBook Pro M1 equipped with a GPU. Visualization was achieved through a combination of Streamlit, Matplotlib, Seaborn, and Plotly Python packages. The project artifacts and code are available at this GitHub page [55].

B. Results

- ***Cypher Commands***

```
{cypher}
    CREATE (contract:Contract {id: '0x0114622386c1a00686e594c70682d7aa0f8afa29'})

CREATE (llmInsights:LLM_Insights)

CREATE (library:Function {name: 'isContract', visibility: 'internal'})
CREATE (library)-[:DEFINED_IN]->(contract)

CREATE (libraryAddress:Library {name: 'Address'})
CREATE (libraryAddress)-[:PART_OF]->(contract)

CREATE (isContractCodePattern:CodePattern {pattern: 'isContract(address account)
       internal view returns', description: 'Code pattern for checking if an
       address is a contract', vulnerability: 'Access Control Issues'})
CREATE (isContractCodePattern)-[:ASSOCIATED_WITH]->(library)

CREATE (accessControlVulnerability:Vulnerability {name: 'Access Control Issue',
       description: 'Vulnerability allowing unauthorized access', severity: 'High
       ', cweId: 'CWE-284', cveId: 'CVE-2021-1234'})
CREATE (accessControlVulnerability)-[:AFFECTED_BY]->(isContractCodePattern)
CREATE (accessControlVulnerability)-[:MITIGATES]->(mitigationTechnique:
       MitigationTechnique {name: 'Implement Access Control'})MATCH (c:Contract {
       address: '0x0114622386c1a00686e594c70682d7aa0f8afa29'}) MERGE (l:Label {
       name: 'Unknown_Label_38'}) MERGE (c)-[:HAS_LABEL]->(l)
```

Above are the Cypher commands generated by LLM from the given preprocessed smart contract code, which is used to build the LLMGraphAgent.

1) Vulnerability Identification on Human Generated code: Figure 4 visualizes the labels created by LLMSmartSec for the code with ground truth labels represented on the x-axis. We observe that LLMs can label the code with specific vulnerabilities, which is more accurate than human-labeled vulnerabilities. It also labeled major vulnerabilities as 'access control vulnerabilities' with an accurate description of how they can be exploited and mitigated. The number on top of each bar represents the total count of smart contracts with the label on the x-axis, and each color coding on the bar represents the vulnerability identified by LLM. We use OpenAI GPT models for all our LLM tasks. We have also tested the base models of LLAMA2 [56] and Claude APIs [57] but these failed to accurately identify the specific vulnerabilities, frequently misclassifying many as 'Reentrancy'.

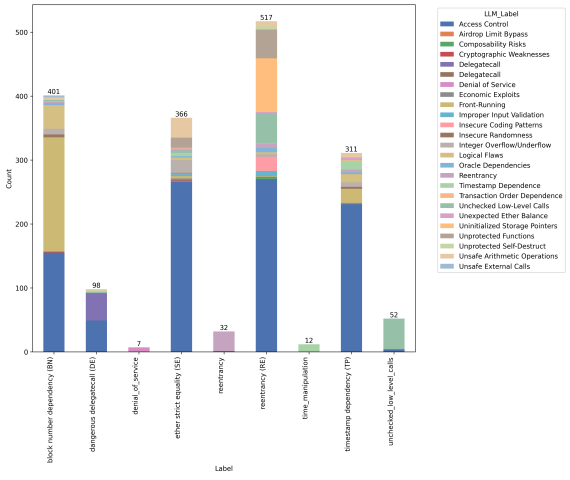


Fig. 4: Vulnerabilities Identified by LLM

- ***Solidity Code with Front Running Vulnerability***

```
pragma solidity ^0.4.22;
\Solidity
/* @source: https://github.com/sigp/solidity-security-blog
 */

contract FindThisHash {
    bytes32 constant public hash = 0
            xb5b5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a;

    constructor() public payable {} // load with ether

    function solve(string solution) public {

        require(hash == sha3(solution));
        msg.sender.transfer(1000 ether);
    }
}
```

The FindThisHash contract, shown above, was utilized to evaluate the effectiveness of our LLMSmartSec approach in accurately detecting vulnerabilities. The chosen Solidity contract contains one of the critical vulnerability known as 'Front Running'.
Below is a snapshot from the audit report detailing the detected vulnerabilities and corresponding recommendations for the FindThisHash contract review. While the report does not explicitly label the issue as 'Front Running,' it thoroughly analyzes the code's impact along with suggested fixes.

LLM Response: Main Findings (Extract from Audit Report)

- Vulnerability 1: Without a limit on attempts or gas usage, an attacker could utilize excess computational resources by repeatedly calling the solve function with incorrect solutions, causing a waste of gas.
- Vulnerability 2: sha3 is an alias to keccak256 in older versions of Solidity. However, starting from Solidity 0.5.0, sha3 was removed and only keccak256 is used. This can cause errors if the contract were to be mistakenly compiled with a newer compiler.
- Vulnerability 3: If the contract balance never reaches 1000 ether or more, the ether sent to this contract is effectively frozen and unusable since the only way to withdraw is by solving the hash, which triggers a transfer of 1000 ether.
- Recommendations/Fixes: : Adjust the transfer amount to a dynamic value equivalent to the contract's balance (or a safely defined withdrawal limit) to prevent transferring more ether than the contract possesses.

Below is the snapshot of LLM's response from the perspective of a Developer, an Ethical Hacker, and an Auditor.

LLM Response: Three Views of LLM (Extract from Audit Report)

- ***Developer Perspective***
  The FindThisHash contract presents a simple puzzle where participants must find the input string that matches a given hash when hashed. If the correct solution is provided, the participant is rewarded with 1000 ether. The contract ensures that the reward is refunded during deployment.
- ***Ethical Hacker Perspective***
  Several vulnerabilities were identified, particularly: The contract is susceptible to brute force attacks as there are no rate limitations or proof-of-work mechanisms. The usage of sha3, which is an outdated keyword in Solidity, keccak256 should be used instead. Lack of event logging in the solve function, which limits the oversight of solution attempts.
- ***Auditor Perspective***
  From an auditor's perspective, the contract has significant vulnerabilities in its design, particularly the lack of protection against brute force attacks and the use of outdated hash functions. These issues significantly affect the contract's reliability and security.

2) Vulnerability Identification on LLM Generated code: We have asked LLM to generate a sample code with hidden vulnerabilities to perform this analysis. Below is the code (VulnerableContract) generated by LLM. This evaluation assesses the ability of our approach to detect vulnerabilities in LLM-generated code.

- ***Solidity Code written by LLM with several vulnerabilities and potential issues***

```
pragma solidity ^0.8.0;

contract VulnerableContract {
    address public owner;
    mapping (address => uint256) private balances;

    constructor() {
        owner = msg.sender;
    }

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function transfer(address _to, uint256 _amount) public {
        require(balances[msg.sender] >= _amount, "Insufficient balance");
        balances[msg.sender] -= _amount;
        balances[_to] += _amount;
    }

    function withdrawAll() public {
        uint256 amount = balances[msg.sender];
        payable(msg.sender).transfer(amount);
        balances[msg.sender] = 0;
    }
```


Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

```
26
27      function updateOwner(address _newOwner) public {
28          if (tx.origin != msg.sender) {
29              owner = _newOwner;
30          }
31      }
32
33      function getBalance(address _account) public view returns (uint256) {
34          return balances[_account];
35      }
36  }
```

Below is the snapshot from the report generated for VulnerableContract code with high, medium, and low vulnerabilities with recommendations/fixes for each.

LLM Response: Main Findings (Extract from Audit Report)

- Critical Vulnerability: Reentrancy Attack Risk: High Function: withdrawAll Issue: Transfers funds before updating balances. Impact: Multiple calls by a malicious recipient can drain the account balance. Recommendation: Employ reentrancy guard.
  Ownership Hijacking via tx.origin Risk: High Function: updateOwner Issue: An attacker can craft a chain of contract calls to exploit tx.origin. Impact: Unauthorized takeover of the contract ownership. Recommendation: Check ownership using msg.sender.
- Medium Vulnerability:
  Unchecked Arithmetic Operations: Risk: Medium Function: deposit Issue: Adding values to balances can lead to overflows. Impact: Possible incorrect balances (although less of an issue in Solidity 0.8.0 due to built-in checks). Recommendation: Explicitly use checked arithmetic for clarity.
- Low Vulnerability: Gas Optimization: Risk: Low Issue: Inefficient balance updates. Impact: Higher gas costs for frequent transactions. Recommendation: Reduce redundant storage operations.

Below is the snapshot of the perspective of LLMDev, LLMeHack, and LLMAudit for the VulnerableContract code.

LLM Response: Three Views of LLM (Extract from Audit Report

- ***Developer Perspective***
  Basic deposit/withdrawal mechanics implemented.
  Simple transfer functionality between users.
  Allows owner updates conditional on the transaction origin.
  Review Comments: deposit function correctly increments balances.
  transfer function properly checks for sufficient balance but doesn't account for reentrancy.
  withdrawAll function doesn't guard against reentrancy, posing significant risk.
  updateOwner logic flawed due to tx.origin dependency.
  Code Execution Paths:
  Normal deposits and withdrawals function as intended.
  Owner update function contains logic that may lead to incorrect owner updates if exploited.
- ***Ethical Hacker Perspective***
  Reentrancy: Function: withdrawAll Explanation: No reentrancy guard, allowing attackers to drain funds before state updates.
  tx.origin Exploit: Function: updateOwner Explanation: Dependency on tx.origin can enable ownership hijacking through a crafted contract call.
- ***Auditor Perspective***
  Architecture: The contract exhibits a straightforward design with a focus on basic fund handling operations. Yet, its simplicity introduces certain attack vectors due to inadequate security checks. Relies solely on Solidity's native structures without external libraries, which both simplifies the audit and makes the noted vulnerabilities plausible due to lack of protective measures offered by standard libraries like OpenZeppelin.
  The contract requires substantial improvements in both security and operation efficiency. Anomalies like the usage of tx.origin suggest the need for an enhanced understanding of Solidity security implications by the developer.

C. Comparison with existing works

To assess the effectiveness of our proposed approach, we compared the auto-generated audit reports produced by LLMSmartSec with existing human-generated reports. While auto-generated reports provide a valuable starting point, they may still need the same level of credibility as human experts in identifying all potential vulnerabilities in smart contracts. By benchmarking LLMSmartSec's performance against human audits, we can evaluate its strengths and limitations and identify areas for further improvement. We conducted a comparative analysis between LLMSmartSec's auto-generated reports and human-audited reports across the following key aspects of smart contract security analysis:

1) *Vulnerabilities:*
   Ability to detect vulnerabilities with detailed assessment of the impact on smart contract's security and integrity
2) *Potential risks:*
   Ability to identify issues that may not be vulnerabilities yet but could lead to security risks if left unaddressed.
3) *Recommendations/Fixes:*
   Ability to generate information and code snippets that Suggest actionable measures to mitigate identified vulnerabilities and potential risks

To perform this comparison, we used the Ethereum blockchain project 'Wise Lending' [58] which is in the recently audited projects list of Code4rena. Wise Lending contains 44 solidity files (considered a large project), and the time spent on manual audits was 41 hours, whereas the LLMSmartSec audits were 36.76 minutes. We selected Code4rena because it offers a copy of the audited code along with a detailed report for comparison.

Table III summarizes the comparison observation between human audit reports and the report generated from our approach. The stats demonstrate the efficiency of our approach in providing detailed vulnerabilities that are split into wider categories and much more detailed recommendations/fixes in comparison to the human-audited report. Auditing the entire project using our approach was completed within an hour. The comparison of the human reports with the generated reports was manually made due to the choice of words being different for both approaches.

TABLE III: Comparison with existing works

Work	Vulnerabilities	Potential risks	Fixes	Time
Project Reviewed: Wise Lending [58]				
Code4rena [59]	12	8	15	41 Hrs
Our Work	25	21	29	36.76 Mins

We tried comparing our approach with existing LLM-based research, but the common metrics like accuracy and precision were not reproducible. Each LLM has its own language and can not generate the exact labeled vulnerabilities. However, the presented comparison with human-generated audit reports, which are the industry standard, shows that our results are practical and effective.

V. Conclusion and Future Work

In this research, we explored multi-view AI assistants to accurately and cost-effectively identify and fix smart contract vulnerabilities. We used OpenAI tools to analyze smart contract code and build an annotated Control Flow Graph (CFG), capturing insights from LLMs for future code analysis reuse. Additionally, we automated the smart contract audit report creation process and compared the auto-generated reports with manually generated ones. For future research, we aim to enhance the annotated CFG to identify more complex patterns in smart contract vulnerabilities. We will continue developing the LLMGraphAgent to maintain up-to-date knowledge of vulnerabilities and their fixes. Furthermore, we will investigate adapting our multi-view AI assistant approach to Vyper, Rust, and JavaScript languages. This will involve studying each language's specific syntax, semantics, and vulnerabilities, and modifying the LLMGraphAgent's knowledge base and analysis techniques accordingly.

References

[1] P. Cuccuru, "Beyond bitcoin: an early overview on smart contracts," *International Journal of Law and Information Technology*, vol. 25, no. 3, pp. 179–195, 2017.
[2] M. E. Peck, "Blockchains: How they work and why they'll change the world," *IEEE spectrum*, vol. 54, no. 10, pp. 26–35, 2017.
[3] D. A. Zetzsche, D. W. Arner, and R. P. Buckley, "Decentralized finance (defi)," *Journal of Financial Regulation*, vol. 6, pp. 172–203, 2020.
[4] C. R. Harvey, A. Ramachandran, and J. Santoro, *DeFi and the Future of Finance*. John Wiley & Sons, 2021.
[5] T. A. Xu and J. Xu, *A Short Survey on Business Models of Decentralized Finance (DeFi) Protocols*. Springer International Publishing, 2023, p. 197–206.
[6] A. Trozze, B. Kleinberg, and T. Davies, "Detecting defi securities violations from token smart contract code," *arXiv preprint arXiv:2112.02731*, 2021.
[7] Y. Huang, T. Zhang, S. Fang, and Y. Tan, "Deep smart contract intent detection," *arXiv preprint arXiv:2211.10724*, 2022.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.

[8] "Understanding a revolutionary and flawed grand experiment in blockchain: The dao attack," *Social Science Research Network*, 2017.

[9] F. B. Insights, "Smart contracts market," 2024. [Online]. Available: https://www.fortunebusinessinsights.com/smart-contracts-market-108635

[10] A. Rozario and M. Vasarhelyi, "Auditing with smart contracts," *The International Journal of Digital Accounting Research*, vol. 18, pp. 1–27, 01 2018.

[11] I. David, L. Zhou, K. Qin, D. Song, L. Cavallaro, and A. Gervais, "Do you still need a manual smart contract audit?" 2023.

[12] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. ACM, Nov. 2023. [Online]. Available: http://dx.doi.org/10.1145/3576915.3623175

[13] T. Ahmed and P. Devanbu, "Few-shot training llms for project-specific code-summarization," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023.

[14] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," 2023.

[15] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20. ACM, Jun. 2020.

[16] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, "Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications," 2024.

[17] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," 2023.

[18] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," 2023.

[19] S. Xia, S. Shao, M. He, T. Yu, L. Song, and Y. Zhang, "Auditgpt: Auditing smart contracts with chatgpt," 2024.

[20] S. Wang, K. Pei, and J. Yang, "Smartinv: Multimodal learning for smart contract invariant inference," in *2024 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2024, pp. 129–129.

[21] R. M. Parizi, A. Dehghantanha, K.-K. R. Choo, and A. Singh, "Empirical vulnerability analysis of automated smart contracts security testing on blockchains," ser. CASCON '18. USA: IBM Corp., 2018, p. 103–113.

[22] S. Chaliasos, M. A. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, and B. Livshits, "Smart contract and defi security tools: Do they meet the needs of practitioners?" in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24, 2024.

[23] K. Qin, Z. Ye, Z. Wang, W. Li, L. Zhou, C. Zhang, D. Song, and A. Gervais, "Towards automated security analysis of smart contracts based on execution property graph," 2023.

[24] F. Özdemir Sönmez and W. J. Knottenbelt, "Contractarmor: Attack surface generator for smart contracts," *Procedia Computer Science*, vol. 231, pp. 8–15, 2024, 14th International Conference on Emerging Ubiquitous Systems and Pervasive Networks / 13th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare (EUSPN/ICTH 2023).

[25] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "Propertygpt: Llm-driven formal verification of smart contracts through retrieval-augmented property generation," 2024.

[26] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, M. Shi, and Y. Liu, "Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning," 2024.

[27] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, May 2019. [Online]. Available: http://dx.doi.org/10.1109/WETSEB.2019.00008

[28] C. F. Torres, M. Steichen, and R. State, "The art of the scam: demystifying honeypots in ethereum smart contracts," in *Proceedings of the 28th USENIX Conference on Security Symposium*, ser. SEC'19. USA: USENIX Association, 2019, p. 1591–1607.

[29] I. Nikolic, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, "Finding the greedy, prodigal, and suicidal contracts at scale," 2018.

[30] D. Weber, A. Ibrahim, H. Nemati, M. Schwarz, and C. Rossow, "Osiris: Automated discovery of microarchitectural side channels," 2021.

[31] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "Smartcheck: Static analysis of ethereum smart contracts," in *2018 IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2018, pp. 9–16.

[32] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 254–269. [Online]. Available: https://doi.org/10.1145/2976749.2978309

[33] "Mythril," 2024. [Online]. Available: https://github.com/Consensys/mythril

[34] *Securify: Practical Security Analysis of Smart Contracts*, 2018.

[35] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A user-friendly symbolic execution framework for binaries and smart contracts," 2019.

[36] Z. Wang, W. Wu, C. Zeng, J. Yao, Y. Yang, and H. Xu, "Graph neural networks enhanced smart contract vulnerability detection of educational blockchain," 2023.

[37] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural network," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, C. Bessiere, Ed. International Joint Conferences on Artificial Intelligence Organization, 7 2020, pp. 3283–3290, main track. [Online]. Available: https://doi.org/10.24963/ijcai.2020/454

[38] G. Ayoade, E. Bauman, L. Khan, and K. Hamlen, "Smart contract defense through bytecode rewriting," in *2019 IEEE International Conference on Blockchain (Blockchain)*, 2019, pp. 384–389.

[39] J. Li, G. Lu, Y. Gao, and F. Gao, "A smart contract vulnerability detection method based on multimodal feature fusion and deep learning," *Mathematics*, vol. 11, no. 23, 2023. [Online]. Available: https://www.mdpi.com/2227-7390/11/23/4823

[40] G. A. Pierro, "Smart-graph: Graphical representations for smart contract on the ethereum blockchain," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2021, pp. 708–714.

[41] H. Wu, Z. Zhang, S. Wang, Y. Lei, B. Lin, Y. Qin, H. Zhang, and X. Mao, "Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*, 2021, pp. 378–389.

[42] ——, "Peculiar paper code." [Online]. Available: https://github.com/wuhongjun15/Peculiar

[43] X. Tang, Y. Du, A. Lai, Z. Zhang, and L. Shi, "Deep learning-based solution for smart contract vulnerabilities detection," *Scientific Reports*, vol. 13, 11 2023.

[44] C. Ma, S. Liu, and G. Xu, "Hgat: smart contract vulnerability detection method based on hierarchical graph attention network," *Journal of Cloud Computing*, vol. 12, 06 2023.

[45] M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023)*, 2023, to appear.

[46] C. Sendner, R. Zhang, A. Hefter, A. Dmitrienko, and F. Koushanfar, "G-scan: Graph neural networks for line-level vulnerability identification in smart contracts," 2023.

[47] R. I. T. Jensen, V. Tawosi, and S. Alamir, "Software vulnerability and functionality assessment using llms," 2024.

[48] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, Y. Wang, X. Lin, T. Chen, and Z. Zheng, "When chatgpt meets smart contract vulnerability detection: How far are we?" 2023.

[49] C. Shou, J. Liu, D. Lu, and K. Sen, "Llm4fuzz: Guided fuzzing of smart contracts with large language models," 2024.

[50] M. Rossini, "Slither audited smart contracts dataset," 2022.

[51] E. E. Alliance, "Eea ethtrust security levels specification version 2," 2024. [Online]. Available: https://entethalliance.org/specs/ethtrust-sl/v2/

[52] S. Arora, "Smart contract audit reports dataset," 2024. [Online]. Available: https://www.kaggle.com/datasets/shivam017arora/smart-contract-audit-reports-dataset

[53] T. B. B. et.all, "Language models are few-shot learners," 2020.

[54] O. et.al, "Gpt-4 technical report," 2024.

[55] dslmllab, "Llmsmartsec," 2024. [Online]. Available: https://github.com/dslmllab/LLMSmartSec

[56] "Llama2," 2024. [Online]. Available: https://llama.meta.com/llama2/

[57] Anthropic, "Claude.ai," 2024. [Online]. Available: https://www.anthropic.com/research

[58] code 423n4, "Wise lending audit details," 2024. [Online]. Available: https://github.com/code-423n4/2024-02-wise-lending/tree/main/contracts

[59] Code4rena, "Code4rena," 2024. [Online]. Available: https://docs.code4rena.com/roles/wardens

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 12:15:58 UTC from IEEE Xplore. Restrictions apply.